Industrial Machine Vibration Analysis Using Digital Signal Processing

Puli Pragnan

Department of Electronics and Communication Engineering

Chaitanya Bharathi Institute of Technology, Hyderabad, India

Abstract A change in a machine's vibration spectrum can be an early sign of a developing fault. This undergraduate project presents a MATLAB implementation that demonstrates how basic digital signal processing (DSP) makes such a change visible. The script generates, in MATLAB, a synthetic "healthy" signal (30 and 60 Hz components) and a synthetic "faulty" signal (the same components plus illustrative 180 and 300 Hz components), adds white Gaussian noise, smooths both signals with a 5-point moving-average (MA) filter, computes single-sided FFT amplitude spectra, and compares the two cases through their spectra, their RMS values, and their spectral energy in a 150–220 Hz band. **All signals are MATLAB-generated synthetic signals; no real industrial machine was recorded or tested, and the work does not show that the approach works on real machines.** The script contains no threshold, classifier or automatic decision rule, and it is not a real-time system; it produces three figures and prints the RMS value and band energy of each signal. The script was executed in MATLAB R2025b and the three resulting figures were observed. In that run, the 150–220 Hz energy of the filtered faulty signal was 0.1894, compared with 0.0278 for the filtered healthy signal (about 6.8 times), while the RMS values were 0.8356 and 0.7836 (a difference of about 6.6%). The MA filter also attenuates the fault components; its theoretical gains are 0.708 at 180 Hz and 0.312 at 300 Hz. The methods are standard textbook techniques, and no new algorithm is claimed.

Keywords: vibration monitoring, condition monitoring, synthetic signals, moving-average filter, FFT, spectral band energy, MATLAB

1 Introduction

Rotating machines such as motors, pumps and fans are central to industry. When a fault develops (unbalance, misalignment, looseness, wear), the machine's vibration usually changes, often before the machine fails, so measuring vibration is a standard way to monitor machine condition [1]. Standards such as ISO 20816-1 give general guidelines for measuring and evaluating machine vibration [2].

A raw vibration waveform is difficult to judge by eye because it mixes noise with several frequencies. DSP helps in two ways: a filter can reduce part of the noise, and the Fourier transform shows which frequencies are present and how strong they are [3,4]. This project applies these two tools in the simplest way so that every step can be run, understood and explained. It is a teaching demonstration on synthetic data, not a validated monitoring system.

2 Problem Statement

A noisy vibration waveform is hard to interpret directly. The problem addressed here is to show, on controlled synthetic signals, how a simple filter and the FFT make a change in frequency content visible, and how a small number of numerical features (RMS value and band energy) respond to that change.

3 Objectives

1. Generate a synthetic healthy signal and a synthetic faulty signal that contains additional fault-related frequency components.
2. Add random noise and reduce part of it with a moving-average filter.
3. Plot the signals before and after filtering, and compute their single-sided FFT spectra.
4. Compare the healthy and faulty filtered spectra, and compute two simple features: the RMS value and the spectral energy in a 150–220 Hz band.
5. State clearly what the script does not do (Section 9.2).

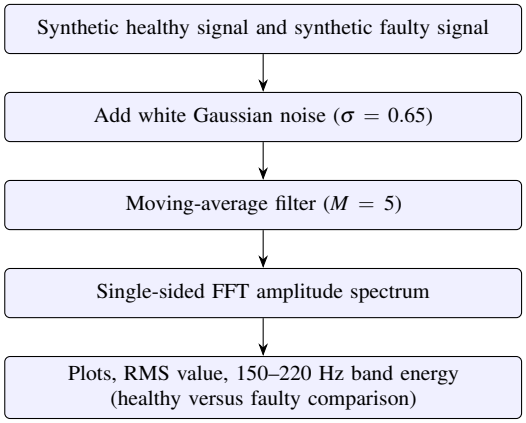


Figure 1: Block diagram of the processing chain in the MATLAB script.

4 Proposed Solution and System Block Diagram

Fig. 1 shows the processing chain implemented in the script. The same chain is applied to the healthy and the faulty signal, so any difference between their outputs comes from the signals and not from the processing.

From input to output.

1. *Signals.* A healthy signal and a faulty signal are generated synthetically (Section 5.1).

2. *Noise.* Independent white Gaussian noise is added to each signal.
3. *Filtering.* Each noisy signal is smoothed with a 5-point moving average.
4. *Spectrum.* The single-sided FFT amplitude spectrum is computed for each signal before and after filtering.
5. *Comparison.* The filtered healthy and faulty spectra are plotted together, and the RMS value and the energy in the 150–220 Hz band are printed for both filtered signals.

The output is three figures and four printed numbers (two features for each of the two signals). The script makes no NORMAL/FAULT decision.

5 DSP Methodology

5.1 Synthetic signals

The sampling frequency is $f_s = 2000$ Hz and the duration is $T = 2$ s, giving $N = 4000$ samples. The healthy signal is the sum of a 30 Hz component and a 60 Hz harmonic (amplitudes 1 and 0.30). The faulty signal adds two illustrative fault-related components at 180 and 300 Hz (amplitudes 0.55 and 0.30). These frequencies and amplitudes are chosen for illustration; they are not taken from a real machine. All components have zero phase and fixed amplitude (Eq. 1).

5.2 Why a moving-average filter?

Measurement noise changes quickly from sample to sample, while the machine components change smoothly. Averaging M neighbouring samples keeps the slow part and reduces much of the fast random part. It is one of the simplest low-pass filters and has a single parameter ($M = 5$ here). Its cost is that it also weakens higher frequencies (Table 1); a fault component at a high frequency is reduced along with the noise.

5.3 Why the FFT?

In the waveform (Fig. 2) the components and the noise are added together. The FFT separates them by frequency, so the strength at each frequency can be read directly, and a fault that creates new frequency content shows up as new spectral energy (Figs. 3 and 4).

5.4 A property of any linear filter

A linear filter multiplies the signal and the noise at the same frequency by the same gain $|H(f)|$, so it cannot improve the signal-to-noise ratio at that frequency. It helps only by reducing noise at frequencies where there is little or no signal. In the 150–220 Hz band used here, the mean of $|H(f)|^2$ is 0.481 (the factor applied to the noise energy), while the 180 Hz fault component is scaled by $|H(180)|^2 = 0.501$. These factors are comparable, so the filter should not be expected to increase the contrast between healthy and faulty band energy (Section 8).

6 Mathematical Equations

Synthetic signals. For $n = 0, \ldots, N-1$,

$$x_h[n] = \sin\!\left(2\pi \tfrac{30n}{f_s}\right) + 0.30\sin\!\left(2\pi \tfrac{60n}{f_s}\right),$$
$$x_f[n] = x_h[n] + 0.55\sin\!\left(2\pi \tfrac{180n}{f_s}\right) + 0.30\sin\!\left(2\pi \tfrac{300n}{f_s}\right). \tag{1}$$

Here $f_s = 2000$ Hz and $N = f_s T = 4000$; amplitudes are in arbitrary units.

Noise. Let $x[n]$ denote either $x_h[n]$ or $x_f[n]$. Then

$$x_{\text{noisy}}[n] = x[n] + v[n], \qquad v[n] \sim \mathcal{N}(0, \sigma^2). \tag{2}$$

$v[n]$ is artificial white Gaussian noise with $\sigma = 0.65$ (MATLAB `randn`); the healthy and faulty signals receive independent noise realizations.

Moving-average filter.

$$y[n] = \frac{1}{M} \sum_{m=-(M-1)/2}^{(M-1)/2} x_{\text{noisy}}[n-m], \qquad M = 5, \tag{3}$$

$$|H(f)| = \frac{|\sin(\pi f M/f_s)|}{M\,|\sin(\pi f/f_s)|}. \tag{4}$$

The window length M is odd, so the filter is centred and adds no delay. The code uses `conv(...,'same')`, which treats samples outside the record as zero, so the first and last two output samples are affected by this edge effect. In Eq. (4), $|H(f)|$ is the gain at frequency f; it equals 1 at 0 Hz, decreases with f up to $f_s/M = 400$ Hz, where it is zero (it is also zero at the other multiples of 400 Hz). For white noise, the output noise power is σ^2/M, an improvement of $10\log_{10} M = 6.99$ dB if the signal is not attenuated.

Single-sided FFT amplitude spectrum.

$$X[k] = \sum_{n=0}^{N-1} y[n]\, e^{-j2\pi kn/N}, \qquad f_k = k\frac{f_s}{N}, \tag{5}$$

$$A[k] = c_k \frac{|X[k]|}{N}, \quad c_k = \begin{cases} 1, & k = 0,\ N/2, \\ 2, & k = 1, \ldots, N/2-1. \end{cases} \tag{6}$$

$A[k]$ is scaled so that a sine of amplitude a shows height a. The bin spacing is $\Delta f = f_s/N = 0.5$ Hz. All four signal frequencies are multiples of 0.5 Hz and fall exactly on FFT bins, so no window is applied in this synthetic example.

Features.

$$E_{\text{band}} = \sum_{k:\,150 \le f_k \le 220} A[k]^2, \qquad \text{RMS} = \sqrt{\frac{1}{N}\sum_{n=0}^{N-1} y[n]^2}. \tag{7}$$

The 150–220 Hz band contains 141 FFT bins and was chosen to contain the synthetic 180 Hz fault component; the 300 Hz component is outside it. Both features are computed on the filtered signals.

7 MATLAB Implementation

7.1 How to run

1. Save the listing (or the supplied file) as `Industrial_Machine_Vibration_DSP.m`.

2. In MATLAB press Run, or type the file name in the Command Window. The script uses only base MATLAB functions (`randn`, `conv`, `fft`, `subplot`, `plot`). The single-sided FFT function is a local function placed at the end of the script file; MATLAB documents local functions in scripts from R2016b onward and, before R2024a, requires them to be at the end of the file, as they are here [5].
3. Three figure windows open and four numbers are printed in the Command Window. The script does not save figures or a log file.
4. The parameters (noise level, window length, band, duration) are set in the first lines of the script.

7.2 Execution status

The script in Appendix A was implemented and executed in MATLAB R2025b (version 25.2, Update 3), and the three figure windows it produces were observed. The signals are generated by the script itself (Section 5.1); they are synthetic, and no recording from a real industrial machine was used. The four numbers in Table 2 are the values printed in the MATLAB Command Window in that run, and Figs. 2–4 are the figures produced by the script in that run, exported from MATLAB without changes to their content. The script seeds the random generator with `rng(7)`, so the noise is intended to be repeatable, but the exact noise-dependent values (RMS values and band energies) are those of the run reported here; a different seed would give somewhat different values. The filter gains in Table 1 are computed analytically from Eq. (4) and do not depend on the random noise. They were not printed by the script.

What was and was not tested. Only this one execution of the script with the settings in the listing was carried out. No other seeds, noise levels, window lengths, signals or machines were tried, and no real vibration data were processed.

7.3 Complete code

The complete script (114 lines) is given in Appendix A (Listing 1), after the references, exactly as executed; it is also delivered as a separate file.

8 Results and Discussion

All results are from synthetic signals generated by the MATLAB script, with artificial noise, and are taken from one execution in MATLAB R2025b (Section 7.2). No real-machine data were used.

8.1 Signals before and after filtering

Fig. 2 shows the first 0.25 s of the healthy and faulty signals before and after the MA filter. Before filtering, both traces are dominated by rapid sample-to-sample noise, with excursions of about ± 3 (read from the axes); the slow oscillation near 30 Hz (about seven to eight cycles in 0.25 s) is visible underneath it. After smoothing, the traces are visibly smoother, the excursions are smaller (about ± 2), and the slow oscillation is much easier to see in both signals. The smoothed faulty trace appears somewhat more irregular on each cycle than the smoothed healthy trace; this is a visual impression only and was not quantified. The vertical axis ranges differ between panels because MATLAB scaled each axis automatically.

Table 1: Gain $|H(f)|$ of the $M = 5$ moving-average filter at the four signal frequencies, computed analytically from Eq. (4) (independent of the random noise; not printed by the script).

Frequency (Hz)	30	60	180	300
Role	machine	machine	fault	fault
$\lvert H(f)\rvert$	0.991	0.965	0.708	0.312

Table 2: Values printed in the MATLAB R2025b Command Window by the script for the filtered signals (synthetic data; one noise realization, `rng(7)`).

Quantity	Healthy	Faulty
Filtered RMS	0.7836	0.8356
150–220 Hz spectral energy	0.027807	0.189427

8.2 Spectra

Fig. 3 shows the spectra before and after filtering. In the healthy case there are two clear peaks, at 30 Hz (magnitude close to 1) and at 60 Hz (about 0.3), on a low broadband noise floor. In the faulty case the same two peaks are present, and two additional peaks appear at 180 and 300 Hz, which the healthy spectrum does not have. Before filtering, the faulty-case peaks at 180 and 300 Hz reach about 0.57 and 0.3; after filtering they are lower, at about 0.4 and 0.09 (values read approximately from the plots). The 300 Hz peak is reduced proportionally much more than the 180 Hz peak, as expected from Table 1. The broadband noise floor is similar before and after filtering at low frequencies but is clearly lower after filtering at higher frequencies, where it falls close to zero near 400 Hz, consistent with the zero of $|H(f)|$ at $f_s/M = 400$ Hz. Fig. 4 overlays the filtered healthy and filtered faulty spectra; they coincide at 30 and 60 Hz and on the noise floor, and differ at 180 and 300 Hz, where only the faulty spectrum shows peaks.

8.3 Feature comparison

Table 2 lists the four values printed by the script in MATLAB. The 150–220 Hz band energy of the faulty signal is about 6.8 times that of the healthy signal ($0.189427/0.027807 \approx 6.8$), whereas the RMS value is only about 6.6% higher (0.8356 versus 0.7836). In this run the band energy is therefore a much more sensitive indicator of this particular synthetic fault than the overall RMS value, because the fault adds energy at a few specific frequencies while the machine components at 30 and 60 Hz dominate the RMS.

Consistency check with theory. The band energy can also be estimated analytically. For white noise of standard deviation $\sigma = 0.65$, the expected energy over the $K = 141$ bins before filtering is $4\sigma^2 K/N = 0.0596$; the mean of $|H(f)|^2$ over the band is 0.481, so about 0.0287 is expected after filtering. Adding the filtered 180 Hz component, $(0.55 \times 0.708)^2 = 0.152$, gives about 0.180 for the faulty signal. The MATLAB values, 0.0278 and 0.1894, are within

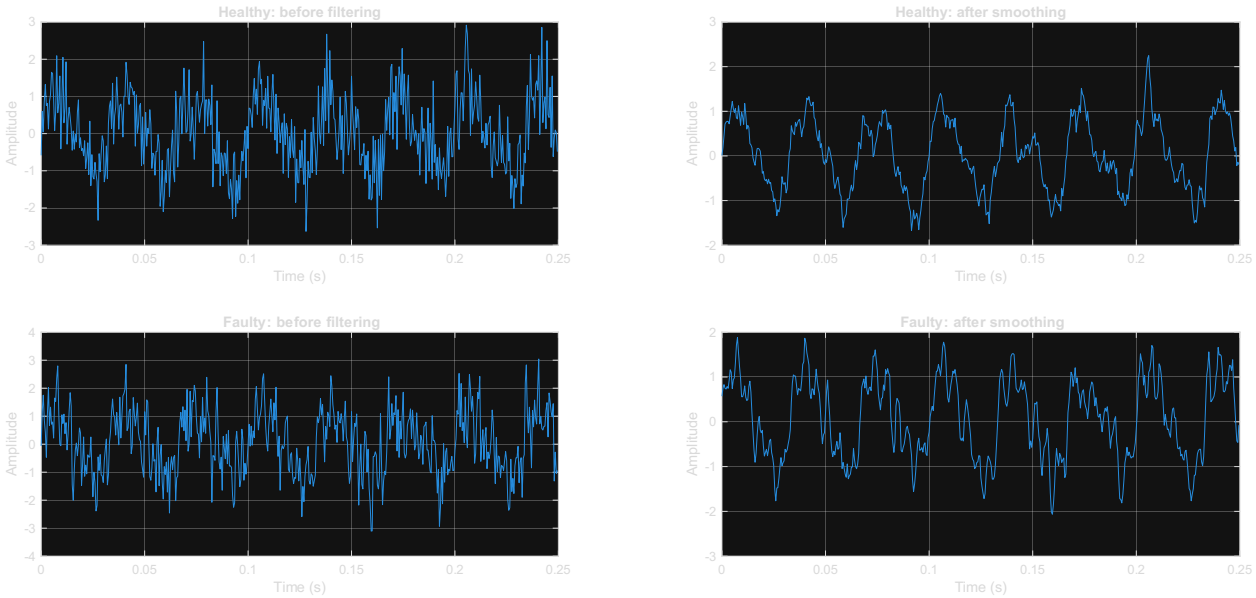


Figure 2: Healthy (top) and faulty (bottom) synthetic vibration signals, before filtering (left) and after the 5-point MA filter (right), first 0.25 s. MATLAB R2025b output of the script in Appendix A.

about 3% and 5% of these estimates. Each printed value comes from a single noise realization and scatters around its expectation, so exact agreement is not expected, and this single run does not by itself show how large that scatter is.

8.4 Effect of the moving-average filter

The filter reduces the broadband noise at higher frequencies (Fig. 3) and the waveform becomes smoother (Fig. 2), but it also reduces the fault components by the factors in Table 1. Because the script does not compute the features without filtering, it provides no numerical comparison of the band energy with and without the filter, so no claim is made here about whether filtering improves the separation between healthy and faulty signals. The linear-filter property of Section 5.4 suggests that it need not.

9 Applications, Limitations and Conclusion

9.1 Applications

In its present form the script serves as a teaching demonstration of noise, filtering, the FFT and spectrum comparison. Applying a similar chain to real recordings, with features chosen from a machine's own fault frequencies, is possible in principle, but it would first require testing and validation on real machine data, which has not been done here.

9.2 Limitations

- **Synthetic data only.** The signals, faults and noise are generated by the MATLAB script. No real machine was recorded or tested, and the approach is not validated on industrial machines.
- **No decision rule and no real-time operation.** The script has no healthy reference set, threshold, classifier, or fault-detection rate. It does not output NORMAL or FAULT, and it does not identify fault type or location. It processes one stored two-second record per run and is not a real-time monitoring system.
- **Single noise realization.** One healthy and one faulty signal are generated per run, with independent noise, so the printed numbers vary with the random seed. The script estimates no statistical spread, and only one execution was carried out.
- **Frequency-specific feature.** The 150–220 Hz band was chosen because the synthetic fault contains a 180 Hz component. The 300 Hz component is not covered, and for real machines the band must be chosen from knowledge of the machine.
- **Moving average attenuates high frequencies.** Fault signatures at several hundred Hz or higher would be weakened by the filter.
- **No window function.** None is used. This is acceptable here because every component falls exactly on an FFT bin, but real recordings generally require a window before the FFT [6].
- **Single execution.** The reported values and figures come from one execution of the script in MATLAB R2025b; no repeated trials, other seeds or parameter sweeps were performed.
- **Not novel.** Moving averaging, the FFT and band-energy features are standard techniques.

9.3 Conclusion

A MATLAB script was implemented and executed (MATLAB R2025b) that generates synthetic healthy and faulty vibration signals, adds noise, smooths them with a 5-point moving-average filter, computes single-sided FFT spectra, and compares the two cases through their spectra, RMS values and 150–220 Hz band energy. In the observed run, the faulty signal showed extra spectral peaks at 180 and 300 Hz and a band energy of 0.1894 against 0.0278 for the healthy signal (about 6.8 times), while the RMS value changed by only about 6.6%. The filter reduced the noise at higher fre-

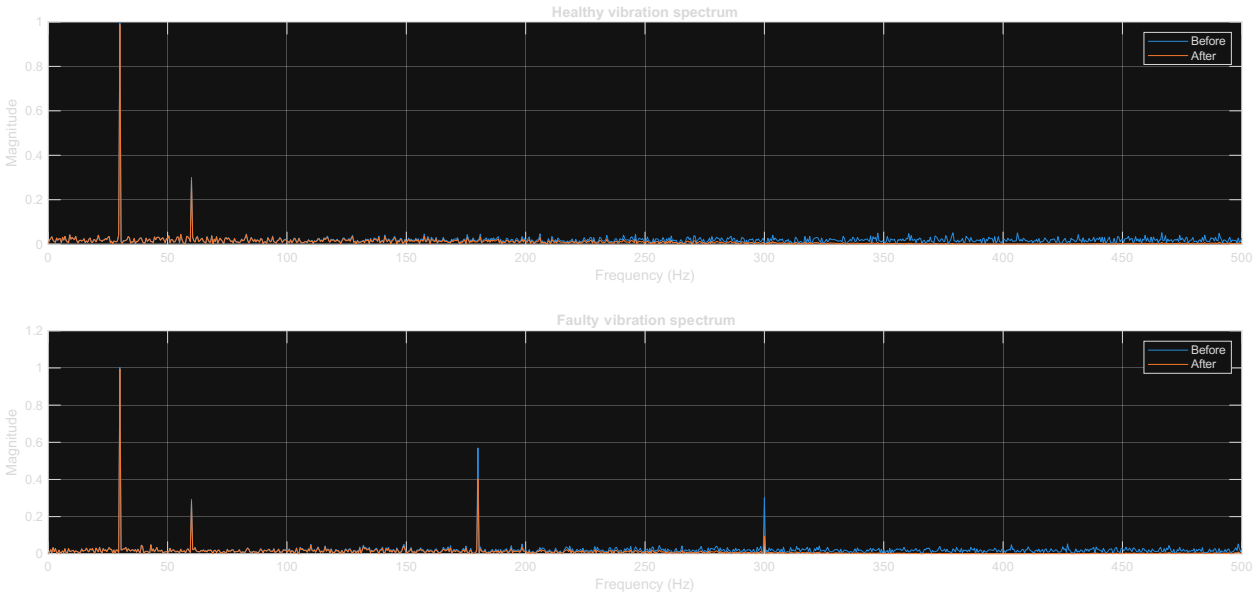


Figure 3: Single-sided FFT amplitude spectra ("Magnitude") before and after MA filtering: healthy (top) and faulty (bottom), 0–500 Hz. MATLAB R2025b output of the script in Appendix A.

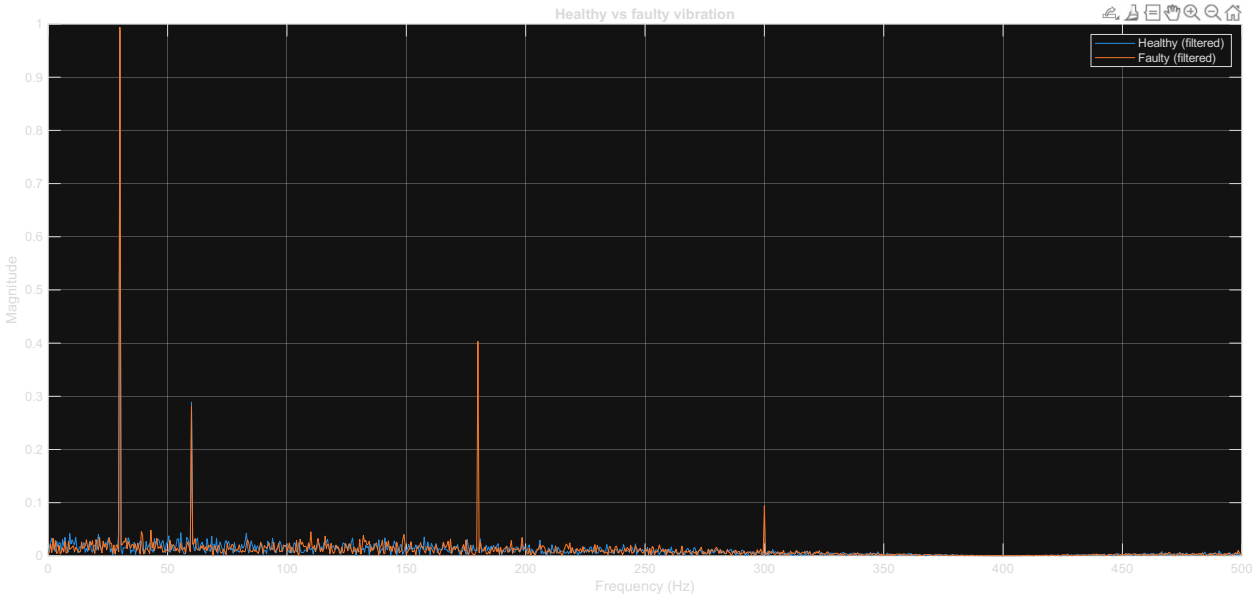


Figure 4: Filtered healthy and filtered faulty spectra, 0–500 Hz. MATLAB R2025b output of the script in Appendix A.

quencies but also attenuated the fault components. These results concern synthetic, MATLAB-generated signals from a single execution only. No real machine was recorded, and no real-time operation, real-machine testing or validated fault detection is claimed. Before any statement about practical performance can be made, the approach must be tested on real machine recordings, with repeated trials and a justified decision rule.

References

[1] R. B. Randall, *Vibration-based Condition Monitoring: Industrial, Aerospace and Automotive Applications*. Chichester, U.K.: Wiley, 2011, ISBN 978-0-470-74785-8.

[2] *Mechanical vibration — Measurement and evaluation of machine vibration — Part 1: General guidelines*, ISO 20816-1:2016, International Organization for Standardization, Geneva, Switzerland, Nov. 2016.

[3] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2010.

[4] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2007.

[5] MathWorks, "Add functions to scripts," MATLAB Documentation. [Online]. Available: https://www.mathworks.com/help/matlab/matlab_prog/local-functions-in-scripts.html. Accessed: Oct. 10, 2026.

[6] F. J. Harris, “On the use of windows for harmonic analysis with the discrete Fourier transform,” *Proc. IEEE*, vol. 66, no. 1, pp. 51–83, Jan. 1978.

A Complete MATLAB Script

The listing below is the file `Industrial_Machine_Vibration_DSP.m` exactly as executed in MATLAB R2025b; it has not been modified for this paper.

Listing 1: `Industrial_Machine_Vibration_DSP.m`: complete MATLAB script (114 lines).

```
% Industrial Machine Vibration Monitoring and Early Fault Detection Using DSP
% Academic demonstration with SYNTHETIC vibration signals (not real machine data).
% Shows vibration waveforms and FFT spectra before/after simple smoothing.

clc; clear; close all;
rng(7);                          % Reproducible random noise

%% 1. Generate synthetic vibration signals
fs = 2000;                       % Sampling frequency (Hz)
T = 2;                           % Duration (seconds)
t = 0:1/fs:T-1/fs;
N = length(t);

% Healthy machine: fundamental vibration and harmonic
healthy_clean = sin(2*pi*30*t) + 0.30*sin(2*pi*60*t);

% Faulty machine: add illustrative fault-related frequency components
faulty_clean = healthy_clean + 0.55*sin(2*pi*180*t) ...
                             + 0.30*sin(2*pi*300*t);

% Add random noise
noise_level = 0.65;
healthy_noisy = healthy_clean + noise_level*randn(size(t));
faulty_noisy  = faulty_clean  + noise_level*randn(size(t));

%% 2. Reduce some rapid fluctuations using a moving-average filter
% Note: smoothing can also weaken high-frequency fault information.
windowLength = 5;
kernel = ones(1, windowLength)/windowLength;
healthy_filtered = conv(healthy_noisy, kernel, 'same');
faulty_filtered  = conv(faulty_noisy, kernel, 'same');

%% 3. Plot signals before and after smoothing
showN = min(N, round(0.25*fs));
figure('Name','Vibration Before and After Filtering','Color','w');

subplot(2,2,1);
plot(t(1:showN), healthy_noisy(1:showN));
title('Healthy: before filtering'); xlabel('Time (s)');
ylabel('Amplitude'); grid on;

subplot(2,2,2);
plot(t(1:showN), healthy_filtered(1:showN));
title('Healthy: after smoothing'); xlabel('Time (s)');
ylabel('Amplitude'); grid on;

subplot(2,2,3);
plot(t(1:showN), faulty_noisy(1:showN));
title('Faulty: before filtering'); xlabel('Time (s)');
ylabel('Amplitude'); grid on;

subplot(2,2,4);
plot(t(1:showN), faulty_filtered(1:showN));
title('Faulty: after smoothing'); xlabel('Time (s)');
ylabel('Amplitude'); grid on;

%% 4. Calculate single-sided FFT spectra
[f, H_noisy] = singleSidedFFT(healthy_noisy, fs);
[~, H_filtered] = singleSidedFFT(healthy_filtered, fs);
[~, F_noisy] = singleSidedFFT(faulty_noisy, fs);
[~, F_filtered] = singleSidedFFT(faulty_filtered, fs);

figure('Name','FFT Before and After Filtering','Color','w');
subplot(2,1,1);
plot(f,H_noisy,'DisplayName','Before'); hold on;
plot(f,H_filtered,'DisplayName','After');
xlim([0 500]); title('Healthy vibration spectrum');
xlabel('Frequency (Hz)'); ylabel('Magnitude');
legend('Location','northeast'); grid on;

subplot(2,1,2);
plot(f,F_noisy,'DisplayName','Before'); hold on;
```

```
plot(f,F_filtered,'DisplayName','After');
xlim([0 500]); title('Faulty vibration spectrum');
xlabel('Frequency (Hz)'); ylabel('Magnitude');
legend('Location','northeast'); grid on;

%% 5. Compare healthy and faulty spectra after filtering
figure('Name','Healthy vs Faulty Spectrum','Color','w');
plot(f,H_filtered,'DisplayName','Healthy (filtered)'); hold on;
plot(f,F_filtered,'DisplayName','Faulty (filtered)');
xlim([0 500]); title('Healthy vs faulty vibration');
xlabel('Frequency (Hz)'); ylabel('Magnitude');
legend('Location','northeast'); grid on;

%% 6. Simple feature comparison
% The 150-220 Hz band is illustrative because the synthetic fault includes
% a 180 Hz component. For real data, choose features from the machine.
band = (f >= 150 & f <= 220);
healthyRMS = sqrt(mean(healthy_filtered.^2));
faultyRMS = sqrt(mean(faulty_filtered.^2));
healthyBandEnergy = sum(H_filtered(band).^2);
faultyBandEnergy = sum(F_filtered(band).^2);

fprintf('\n--- Synthetic demonstration results ---\n');
fprintf('Healthy filtered RMS: %.4f\n', healthyRMS);
fprintf('Faulty filtered RMS : %.4f\n', faultyRMS);
fprintf('Healthy 150-220 Hz spectral energy: %.6f\n', healthyBandEnergy);
fprintf('Faulty 150-220 Hz spectral energy : %.6f\n', faultyBandEnergy);
fprintf('\nThese are synthetic examples, not real-machine results.\n');
fprintf('Moving-average smoothing may also weaken fault-related frequencies.\n');

%% Local function: single-sided FFT using base MATLAB
function [f, magnitude] = singleSidedFFT(x, fs)
    x = x(:).';
    N = length(x);
    X = fft(x);
    magnitude = abs(X/N);
    magnitude = magnitude(1:floor(N/2)+1);
    if length(magnitude) > 2
        magnitude(2:end-1) = 2*magnitude(2:end-1);
    end
    f = fs*(0:floor(N/2))/N;
end
```